UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Universidad Técnica Federico Santa María
Campus Santiago San Joaquín
Bases de Datos Avanzadas
Segundo Semestre 2026

Tarea 3: Neo4j y análisis de grafos

Profesor: José Luis Martí Lara
Ayudante: Eduardo Rodríguez Prado

1. Introducción

Neo4j es una base de datos NoSQL orientada a grafos. La información se representa mediante **nodos**, **relaciones** y **propiedades**. El lenguaje *Cypher* permite consultar patrones de conexión, recorrer relaciones y agrupar resultados. La biblioteca *Graph Data Science* (GDS) permite aplicar algoritmos que analizan la estructura del grafo.

2. Contexto del caso

Durante Halloween, **15 estudiantes** de INF325 salen a pedir dulces, pero cerca de la universidad solo hay **10 casas/lugares** cercanos para pedir dulces. Cada estudiante visita exactamente **5 casas/lugares distintos**, para que les de la noche. El ayudante del ramo les dijo que el que obtuviese más dulces se llevaría un bonus en las tareas, por lo que les pidió que anotaran a que casa fueron y cuantos dulces les dieron. Finalmente el ayudante confecciono una base de datos en Neo4j, donde para obtener los puntos extra los estudiantes deben descubrir quien fue el ganador, o ganadores si hay empate, viendo a que casas visito y cuantos dulces obtuvo.

En Aula se proporcionará el archivo `tarea.dump`, que contiene el grafo completo. **No debe generar, insertar ni modificar los datos del caso.**

Modelo del grafo

- `Estudiante`: su propiedad `id` contiene el nombre del estudiante.
- `Casa`: tiene un `id` numérico y una propiedad `direccion` con su nombre temático.

- `PIDIO_DULCES`: relación dirigida desde un estudiante hacia una casa. Representa una visita, incluso cuando no se recibieron dulces. Incluye las propiedades `cantidad` y `tipo`.

```
(e:Estudiante)-[r:PIDIO_DULCES]->(c:Casa)
```

La propiedad `cantidad` puede tomar los siguientes valores:

Cantidad	Significado	Probabilidad de generación
0	No recibió dulces	20 %
1	Recibió un dulce	67 %
2	Recibió dos dulces	13 %

Estas probabilidades describen cómo se generó la base; los porcentajes observados en las 75 visitas no tienen por qué coincidir exactamente con ellas. Si `cantidad` es 0, `tipo` es `Ninguno`; en los demás casos sera un tipo de dulce. La cantidad total de dulces se obtiene sumando `cantidad`, no contando relaciones.

3. Set Up

1. Instalar Neo4j Desktop y cargar la base

1. Instale Neo4j Desktop, disponible gratuitamente para uso de desarrollo.
2. Descargue `tarea.dump` desde Aula. Cree una instancia local utilizando una versión de Neo4j compatible con el archivo suministrado y con GDS. Registre las versiones utilizadas en el informe.
3. En Desktop 2.x puede crear una instancia desde un respaldo (*Create from backup*) y seleccionar `tarea.dump`. También puede detener una instancia existente y usar su menú de administración, opción *Load database from file*. Los nombres de las opciones pueden variar según la versión de Desktop.
4. Inicie la instancia y seleccione la base cargada en **Query** (Desktop 2.x). Compruebe el nombre de la base seleccionada: no ejecute las consultas sobre `system` ni sobre otra base vacía.

El archivo `.dump` es un respaldo de base de datos; no se carga mediante la herramienta de importación de CSV. **Considere que al conectar alguno de los pasos previos, hasta estar listo para realizar las Querys, puede demorar unos segundos, o minuto, dando error mientras termina de cargar, por lo que re-intente cada paso un par de veces con intervalo de 30 segundos entre cada intento**.

2. Instalar y verificar GDS

Desde la sección *Plugins* de la instancia, instale **Graph Data Science Library** y reinicie la instancia si corresponde. La instalación debe utilizar una versión de GDS compatible con su versión de Neo4j. No se requiere APOC para esta tarea.

En Query o Neo4j Browser ejecute:

```
RETURN gds.version() AS versionGDS;
```

Incluya en el informe la versión obtenida y evidencia de que GDS está disponible.

3. Verificar la carga

Ejecute estas consultas por separado en la base del caso:

```
MATCH (e:Estudiante)
RETURN count(e) AS estudiantes;

MATCH (c:Casa)
RETURN count(c) AS casas;

MATCH (:Estudiante)-[r:PIDIO_DULCES]->(:Casa)
RETURN count(r) AS visitas;
```

Los resultados deben mostrar 15 estudiantes, 10 casas y 75 visitas. Si los valores no coinciden, revise la carga; no cree relaciones adicionales para corregir los conteos.

4. Desarrollo

Elabore un informe PDF que incluya, para cada ejercicio, la consulta utilizada, los resultados relevantes y una explicación breve. Agregue capturas legibles cuando se soliciten. Las consultas deben estar también en `consultas.cypher`, identificadas con comentarios y ordenadas de acuerdo con este enunciado.

Sección	Puntaje
Parte 1: Exploración y consultas Cypher	40
Parte 2: Análisis con GDS	40
Parte 3: Búsqueda de caminos	15
Parte 4: Cierre	5
Total	**100**

Se evaluará tanto la ejecución correcta como la interpretación. Las consultas de apoyo pueden reutilizarse, pero debe completar lo solicitado y explicar sus resultados. Considere

todas las visitas, incluidas las de cantidad 0, salvo que un ejercicio indique explícitamente otro filtro. Cuando se solicite un máximo, identifique **todos los empates**; `LIMIT 1` por sí solo no los resuelve.

Parte 1: Exploración y consultas Cypher — 40 puntos

1. Visualización e interpretación del grafo — 10 puntos

Muestre el grafo completo en la vista gráfica de Query o Neo4j Browser. Incluya una captura legible que permita distinguir estudiantes y casas.

Configure, si es necesario, las etiquetas visibles para mostrar `id` en estudiantes y `direccion` en casas. Verifique los conteos con las consultas de configuración, ya que una casa sin visitas podría no aparecer en esta visualización de relaciones.

Explique qué representa cada tipo de nodo, la dirección de las relaciones y las propiedades de una visita seleccionada. Indique por qué el modelo es un **grafo bipartito**.

2. Casa con más visitas — 10 puntos

Obtenga un ranking de las 10 casas con su identificador, dirección y cantidad de visitas, ordenado de mayor a menor. Incluya casas con cero visitas si las hubiera. Identifique la casa o las casas que comparten el máximo.

Explique por qué una visita con `cantidad = 0` también debe contarse.

3. Estudiante con más dulces — 10 puntos

Obtenga un ranking de los 15 estudiantes según la cantidad total de dulces recibidos. Muestre el nombre, el número de visitas y la suma de `cantidad`. Identifique a todos los estudiantes que obtuvieron el máximo.

Explique la diferencia entre contar visitas y sumar dulces. Indique por qué todos pueden tener cinco visitas y, aun así, obtener totales diferentes.

4. Visitas sin dulces — 10 puntos

Liste las visitas cuya `cantidad` es 0, mostrando el estudiante, la dirección de la casa y el tipo registrado. Obtenga además la cantidad total de visitas sin dulces y su porcentaje respecto de las 75 visitas.

Compare el porcentaje observado con el 20 % utilizado en la generación y explique brevemente por qué pueden diferir. Use aritmética decimal para calcular el porcentaje. Si no existen visitas sin dulces, informe el conteo 0 y el porcentaje 0 % con evidencia de la consulta.

Parte 2: Análisis con GDS — 40 puntos

Una **proyección** es una representación del grafo en memoria sobre la que GDS ejecuta los algoritmos. No crea una segunda base de datos ni modifica los nodos y relaciones originales. En esta parte trabajará sin pesos: cada visita representa una conexión, sin importar cuántos dulces produjo.

1. Proyección del grafo — 10 puntos

Cree una proyección llamada `halloween` que incluya ambas etiquetas de nodos y las relaciones `PIDIO_DULCES` con su dirección original:

```
CALL gds.graph.project(
  'halloween',
  ['Estudiante', 'Casa'],
  {PIDIO_DULCES: {orientation: 'NATURAL'}}
)
YIELD graphName, nodeCount, relationshipCount
RETURN graphName, nodeCount, relationshipCount;
```

Incluya el resultado y compruebe los 25 nodos y las 75 relaciones. Explique para qué sirve proyectar el grafo y qué significa `NATURAL` en este caso. Indique por qué no se necesita proyectar `cantidad` para los análisis sin pesos solicitados.

Si necesita repetir la creación, elimine primero *solo la proyección* anterior mediante `gds.graph.drop`, como se muestra en el cierre. La proyección debe crearse nuevamente si reinicia la instancia y ya no está en el catálogo.

2. Degree Centrality: centralidad de las casas — 15 puntos

Ejecute *Degree Centrality* para calcular cuántas visitas recibe cada casa. Como las relaciones apuntan de estudiantes a casas, use la orientación `REVERSE` sobre la proyección dirigida para calcular el grado de entrada. Puede utilizar como base:

```
CALL gds.degree.stream('halloween', {
  orientation: 'REVERSE'
})
YIELD nodeId, score
WITH gds.util.asNode(nodeId) AS nodo, score
RETURN labels(nodo) AS etiquetas, nodo.id AS id, score;
```

Complete la consulta para mostrar **solo casas**, incluyendo su dirección, ordenadas por puntaje descendente. Filtre después de ejecutar el algoritmo, para conservar en la proyección los dos tipos de nodos y sus conexiones.

1. Presente el ranking de las 10 casas e identifique los máximos y empates.

2. Compare los puntajes con el número de visitas de la Parte 1.

3. Explique qué significa tener mayor centralidad en este caso y por qué ello no garantiza haber entregado más dulces.

Pista: con `NATURAL`, el grado de salida de cada estudiante sería 5 y el de cada casa sería 0. Debe justificar por qué se usa `REVERSE` para analizar las visitas recibidas.

3. Node Similarity: estudiantes con visitas parecidas — 15 puntos

Utilice *Node Similarity* con la métrica **Jaccard** para comparar estudiantes según las casas que visitaron. Trabaje sobre la misma proyección dirigida. En este algoritmo se comparan los vecinos de salida: las casas visitadas por cada estudiante. Las casas no tienen vecinos de salida en esta proyección.

El siguiente fragmento entrega los identificadores de los nodos y la similitud:

```
CALL gds.nodeSimilarity.stream('halloween', {
  similarityMetric: 'JACCARD',
  topK: 14,
  similarityCutoff: 0.0001
})
YIELD node1, node2, similarity
RETURN gds.util.asNode(node1).id AS estudiante1,
       gds.util.asNode(node2).id AS estudiante2,
       similarity;
```

El parámetro `topK: 14` permite considerar a los otros 14 estudiantes como candidatos por estudiante. El umbral excluye similitudes nulas. Complete y ordene los resultados para:

1. Mostrar las **5 parejas distintas con mayor similitud**, con sus nombres y puntaje. No repita una pareja como (A, B) y (B, A); puede conservar únicamente las filas donde el primer nombre sea menor que el segundo. Para empates en el quinto lugar, puede ordenar adicionalmente por nombres y mostrar exactamente cinco parejas.

2. Elegir una de esas parejas y mostrar con Cypher las casas que ambos estudiantes visitaron.

3. Calcular manualmente su similitud de Jaccard y compararla con GDS:

$$J(A, B) = \frac{|V(A) \cap V(B)|}{|V(A) \cup V(B)|},$$

donde $V(A)$ y $V(B)$ son los conjuntos de casas visitadas por cada estudiante. Por ejemplo, si comparten 3 casas y cada uno visitó 5, la similitud es $3/(5 + 5 - 3) = 3/7$. Este ejemplo es ilustrativo; debe calcular el valor de la pareja elegida en la base suministrada.

Explique qué representa un resultado cercano a 1 y por qué visitar casas similares no implica haber obtenido la misma cantidad o tipo de dulces.

Parte 3: Búsqueda de caminos — 15 puntos

En esta parte utilice **Cypher**; no es necesario implementar otro algoritmo de GDS. Considere las relaciones sin dirección únicamente para recorrer conexiones. Esto no modifica su dirección almacenada.

1. **Casas compartidas — 5 puntos.** Elija dos estudiantes distintos y obtenga las casas que ambos visitaron. Muestre el patrón completo con los dos estudiantes, las casas y las relaciones. Incluya una captura y explique por qué cada casa compartida produce un camino de longitud 2 entre ellos. Puede reutilizar la pareja analizada en Node Similarity. Si elige una pareja sin casas compartidas, documente ese resultado y seleccione otra que sí comparta.

2. **Camino más corto — 5 puntos.** Obtenga un camino más corto entre los mismos estudiantes. Muestre el camino, su longitud y los nodos recorridos. Puede adaptar el siguiente fragmento; los nombres deben corresponder a estudiantes existentes:

```
MATCH (a:Estudiante {id: 'Camila'}),
      (b:Estudiante {id: 'Diego'})
OPTIONAL MATCH p = shortestPath(
  (a)-[:PIDIO_DULCES*1..24]-(b)
)
RETURN p, length(p) AS longitud, nodes(p) AS recorrido;
```

El límite de 24 relaciones es suficiente para un camino más corto entre nodos distintos de un grafo de 25 nodos. Si hay varios caminos mínimos, basta con mostrar uno. Si no existe camino, `p` será nulo; registre e interprete ese resultado sin agregar conexiones.

3. **Interpretación — 5 puntos.** Explique por qué el recorrido alterna estudiantes y casas, por qué se ignora la dirección al buscar este camino y por qué su longitud cuenta relaciones, no dulces. Aclare que el camino representa conexiones por visitas compartidas: no es una ruta geográfica ni permite saber el orden cronológico de las visitas.

Parte 4: Cierre — 5 puntos

Una vez obtenidas todas las evidencias:

1. **Reflexión — 3 puntos.** En un párrafo, relacione un resultado de Cypher, uno de GDS y uno de caminos. Explique qué información sobre conexiones aporta cada uno y mencione una limitación del modelo.

2. **Liberar la proyección — 2 puntos.** Elimine del catálogo la proyección `halloween` y compruebe que ya no existe:

```
CALL gds.graph.drop('halloween', false)
YIELD graphName
```

```
RETURN graphName;

CALL gds.graph.exists('halloween')
YIELD exists
RETURN exists;
```

Incluya evidencia de `exists = false`. Esta operación libera la proyección en memoria; **no elimine la base original ni sus datos**.

5. Sobre la entrega

1. La tarea se realizará **en parejas**. Pueden buscar compañero mediante el canal habilitado en Discord. **Solo un integrante debe realizar la entrega en Aula**.
2. Entregue un archivo comprimido llamado `apellido1-apellido2.zip`, que contenga:
   - `apellido1-apellido2.pdf`: informe con las respuestas, consultas, resultados, explicaciones y capturas solicitadas.
   - `consultas.cypher`: todas las consultas de verificación, desarrollo, proyección, algoritmos y cierre, en orden de ejecución, separadas por punto y coma e identificadas mediante comentarios.
3. El informe debe incluir **nombre y rol de ambos integrantes**, las versiones de Neo4j y GDS y el nombre de la base utilizada. Puede elaborarse con Word, LaTeX u otra herramienta, pero debe entregarse en PDF.
4. El archivo `consultas.cypher` debe permitir reproducir el trabajo sobre una copia recién cargada de `tarea.dump`. Utilice comentarios `//` para las explicaciones y deje la eliminación de la proyección al final. No incluya contraseñas ni instrucciones para generar, insertar, modificar o borrar los datos originales.
5. No debe volver a entregar `tarea.dump` ni un script generador de datos. Las capturas deben estar incluidas en el PDF y ser legibles.
6. Si utiliza GitHub como material complementario, el repositorio debe ser público y su enlace debe aparecer en el informe. El enlace no reemplaza los dos archivos obligatorios de la entrega en Aula.
7. La entrega será hasta el viernes 6 de noviembre a las 23:59. Los horarios de defensa se comunicarán mediante los canales oficiales del curso.
8. Las consultas pueden realizarse mediante Discord o el foro de tareas en Aula. No se responderán consultas durante las 24 horas previas a la fecha de entrega.

6. Entregable, defensa y cálculo de nota

La evaluación considera el entregable disponible en Aula y una defensa posterior. El desarrollo suma **100 puntos** antes de descuentos.

Entregable

El entregable está compuesto por el informe PDF y el archivo `consultas.cypher`. Se evaluarán la corrección de las consultas, la correspondencia entre resultados y evidencia, y la explicación del grafo y de los algoritmos. Una captura sin consulta o sin interpretación no basta para acreditar todos los requisitos de un ejercicio.

Defensa

Las defensas son **individuales**. Cada integrante debe comprender la totalidad del trabajo. La defensa consistirá en aproximadamente **5 a 10 preguntas cortas**, relacionadas directamente con la tarea. Para cada pregunta se asignará:

- **0 puntos**: respuesta errónea o no responde.
- **0,5 puntos**: respuesta parcialmente correcta.
- **1 punto**: respuesta correcta.

Las preguntas podrán abordar nodos, etiquetas, propiedades, relaciones y su dirección; grafos bipartitos; patrones Cypher; `MATCH` y `OPTIONAL MATCH`; filtros, agregaciones, `count` y `sum`; proyecciones GDS; grado de entrada y salida; Degree Centrality; Node Similarity y Jaccard; caminos más cortos; visitas con cero dulces; interpretación de los resultados y límites del modelo.

Cálculo de nota

La nota final se calculará de la siguiente manera:

$$NF = I \cdot N_D, \qquad N_D = \frac{C_r}{C_t},$$

donde I es la nota obtenida a partir del puntaje del entregable, luego de aplicar los descuentos correspondientes. C_r es la suma de los puntos obtenidos en la defensa, y C_t es la cantidad total de preguntas de esa defensa. Como cada pregunta vale un punto, C_t también es el puntaje máximo de la defensa.

La nota final es individual, aunque el entregable se realice en pareja. **El factor N_D puede ser 0**. Si por falta de tiempo no se alcanzan a responder todas las preguntas previstas, las restantes se consideran con puntaje 0 y forman parte de C_t.

7. Descuentos

Los descuentos de puntaje se aplican a la nota final, obtenida pos-defensa. Los descuentos asociados a la defensa se aplican de forma individual al integrante correspondiente (atraso o falta).

- No respetar el formato PDF del informe o los nombres de archivo solicitados: **15 puntos**.
- No incluir nombre y rol de los integrantes en el informe: **30 puntos**.
- Atraso en Aula: **10 puntos por cada hora o fracción**, aproximando hacia arriba.
- No entregar `consultas.cypher`: **30 puntos**. No existen descuentos por omitir un generador de datos o el respaldo, porque no son entregables de esta tarea.
- Si las imágenes o resultados necesarios para evaluar una sección no son accesibles o legibles, esa sección podrá no ser evaluada. Si usa GitHub, el material requerido debe ser público y accesible.
- Los horarios de defensa se bloquearán el mismo día de la defensa. Solicitar una reagendación después de ese bloqueo tendrá un descuento de **15 puntos**.
- Llegar tarde a la defensa tendrá un descuento de **5 puntos**. Además, el estudiante contará con menos tiempo disponible para responder.

  Por ejemplo, si por falta de tiempo solo se responden 5 de 8 preguntas, las tres preguntas restantes se considerarán con puntaje 0. **No se otorgará tiempo adicional**.
- **No agendar un horario de defensa implica nota 0 inmediata.**
- **No asistir a la defensa implica nota 0 inmediata.**